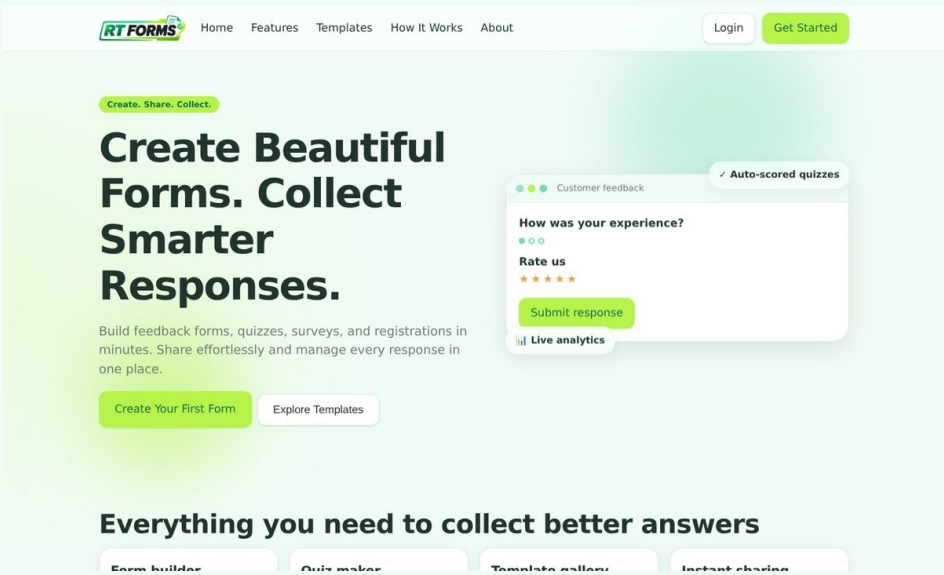
A PERSONAL PROJECT REPORT

RT FORMS

CREATE. SHARE. COLLECT.

Online Form Builder · Quiz Maker · Response Analytics

A Mac-inspired web application built with HTML, CSS and JavaScript



Output — RT FORMS landing page

17	15	8	2	0
QUESTION TYPES	TEMPLATES	CATEGORIES	EXPORT FORMATS	LIBRARIES

Project Title	RT FORMS — Online Form Builder, Quiz Maker and Response Manager
Technologies	HTML5 · CSS3 · JavaScript (ES6+) · localStorage · sessionStorage · GitHub Pages
Project Type	Personal project (front-end only, no backend) — educational demo
Developer ID	RT1504
Live Project	shahrishabh1513-jsk.github.io/rt-forms
Source Code	github.com/shahrishabh1513-jsk/rt-forms
GitHub	github.com/shahrishabh1513-jsk
LinkedIn	linkedin.com/in/rishabh-alpeshabhai-shah

DESIGNED AND DEVELOPED BY

SHAH RISHABH ALPESHBHAI

OVERVIEW

Project at a Glance

RT FORMS is a complete online form-building application. A user starts from a blank form or one of **15 templates**, builds it with **17 question types**, publishes it, shares a link, collects responses and studies them in charts. Quizzes are scored automatically. The whole product is plain HTML, CSS and JavaScript, runs on GitHub Pages and keeps its data in the browser.

FOR FORM CREATORS	FOR RESPONDENTS	FOR DATA OWNERS
• Template gallery with live preview • Editor with autosave • Quiz mode with points • Dashboard with real statistics	• Clean mobile-friendly form page • Progress bar and validation • Confirmation message • Quiz score and explanations	• Response list and details • Distribution charts and averages • CSV and JSON export • Print-friendly report

How It Works

1	2	3	4	5
CHOOSE a template	BUILD questions	PUBLISH and share	COLLECT responses	ANALYSE and export

Technology Stack

HTML5 · CSS3 · JavaScript ES6+ · localStorage · Web APIs · GitHub Pages

Try the Demo

1. Open the live project and choose **Use demo account**.
2. Go to **Templates**, open **Online Quiz**, press **Preview**, then **Use this template**.
3. In the editor press **Publish & Share**, then **Open public form** and submit answers.
4. Return to the dashboard and open **View responses**.

Everything stays in your own browser — nothing is sent to a server. Sample test data is used in the output screens of this report.

Quick Links

Live Project · Project Repository · GitHub Profile · LinkedIn Profile

CONTENTS

Table of Contents

01	Certificate	3
02	Acknowledgement	4
03	Abstract	4
04	Chapter 1 — Introduction	5
05	Chapter 2 — Tools and Technologies	6
06	Chapter 3 — Project Design and Modules	6
07	Chapter 4 — System Implementation	8
08	Chapter 5 — Software Testing	10
09	Chapter 6 — Output Screens	11
10	Chapter 7 — Learning Outcomes	16
11	Chapter 8 — Conclusion and Future Enhancements	16
12	Chapter 9 — References and Glossary	18

CERTIFICATE

Certificate

This is to certify that **SHAH RISHABH ALPESHBHAI** has successfully completed the personal project entitled **"RT FORMS — Online Form Builder with Templates, Quizzes, Sharing and Response Analytics Using HTML, CSS and JavaScript"**.

The project was designed, developed, tested and documented by the candidate using HTML5, CSS3 and Vanilla JavaScript. The work presented in this report is original and has been carried out sincerely and to the best of the candidate's ability. This report is submitted as a record of the candidate's personal project work.

_____
Project Guide
(Name & Signature)
Date: _____

_____
Head of Department
(Name & Signature)
Place: _____

THANKS

Acknowledgement

I would like to express my sincere gratitude to everyone who supported me during the planning, design, development and documentation of my personal project, **RT FORMS**.

I am thankful to my project guide and the faculty members of my department for their guidance, encouragement and constructive suggestions. Their feedback motivated me to focus on clean code, a smooth user journey and a modern, attractive interface.

I also thank my family and friends for their constant support, patience and motivation. I am grateful to the open web community and to the creators of freely available learning resources such as MDN Web Docs, the W3C accessibility guidelines and the GitHub Pages documentation, which made it possible to learn and apply modern web technologies effectively.

Finally, I thank everyone who directly or indirectly helped me in completing this project successfully.

SHAH RISHABH ALPESHBHAI — Developer ID: RT1504

SUMMARY

Abstract

RT FORMS is a modern online form-building platform with the tagline “Create. Share. Collect.” It lets students, teachers, businesses and general users create feedback forms, surveys, registrations, questionnaires and online quizzes without writing code. It is developed with only **HTML5, CSS3 and Vanilla JavaScript (ES6+)** — no framework, build step, backend server or database — and is published as a live website on GitHub Pages.

The application contains a landing page, demo login and registration, a dashboard, a template gallery with detailed previews, a form editor, a respondent-facing form page, a response dashboard and a settings page. The editor supports **17 question types**, and **15 templates** in **8 categories** give every user a quick start, each with its own miniature thumbnail.

Quiz mode supports correct answers, points, a passing score and explanations; choice questions are scored automatically and other types are marked for manual review. Published forms receive a unique ID and a shareable link. The response dashboard shows distribution bars, rating averages, quiz score distribution and daily trends calculated from real stored data, and exports responses as CSV and JSON.

Data is stored in the browser with localStorage behind a small API layer (createForm, getForm, updateForm, publishForm, submitResponse, getResponses, deleteForm), so storage can later be replaced by Supabase or Firebase. The interface follows a macOS-inspired design in a mint green, white and lime palette, with glass navigation, soft shadows, scroll animations and reduced-motion support.

Keywords: RT FORMS · Form Builder · Quiz Maker · HTML5 · CSS3 · JavaScript · localStorage · Templates · Analytics · GitHub Pages

CHAPTER 1 Introduction

1.1 Project Overview

Online forms are used everywhere: classroom feedback, event registration, job applications, customer surveys and quick quizzes. RT FORMS brings the usability of well-known form builders into one original product with its own brand and interface. A user can create a form from scratch or from a template, customise it, preview it as a respondent will see it, publish it, share the link, collect responses and analyse the results. It is a **front-end-first** application: the browser acts as the database, and the code is organised so that a real backend can replace local storage later.

1.2 Problem Statement

- Many form tools require sign-up, hide features behind paid plans or give little control over appearance.
- Teachers and students need reusable templates for feedback, attendance and quizzes instead of rebuilding the same form each time.
- Quizzes need automatic scoring, passing marks and explanations, which basic tools lack.
- Learners rarely see how a complete form builder works: routing, state, validation, scoring, storage, export and analytics.

1.3 Objectives

1. Build a complete form creation, publishing, sharing and response-management application using only HTML, CSS and JavaScript.
2. Provide a template gallery with unique thumbnails and a detailed preview before use.
3. Support 17 question types with the settings and validation each needs.
4. Add quiz mode with points, passing score, automatic scoring and explanations.
5. Give respondents a clean, accessible, mobile-friendly form page.
6. Show real analytics and export data to CSV and JSON.
7. Design a premium macOS-inspired interface in the mint green, white and lime palette.
8. Keep storage behind an API layer so Supabase or Firebase can be added later.

1.4 Scope

Within scope	Outside scope (this version)
Form builder, templates, quiz scoring, public form page, response dashboard, CSV and JSON export, demo authentication, responsive and accessible design	Real accounts and password security, cross-device form links, shared database, e-mail notifications, file storage for uploads, spam protection and rate limiting

A browser-only project cannot show a form to a person on another device, because the data lives in one browser. The application states this clearly instead of pretending otherwise.

1.5 Users and Roles

Role	What they do
Form creator	Signs in (demo), picks or builds a form, configures settings and quiz options, publishes it and reads the responses.
Respondent	Opens the public link, answers, sees validation messages, submits and may see a quiz score.
Developer / learner	Reads the source code, studies the architecture and extends it with a real backend.

CHAPTER 2 Tools and Technologies

Technology	How RT FORMS uses it
HTML5	Semantic landmarks, labelled controls, the native dialog element for accessible modals and details/summary for card menus. One index.html holds the app root, dialog and toast containers.
CSS3	A central block of CSS variables for colours, radii, shadows and transitions; Grid and Flexbox with mobile-first media queries; glass navigation (backdrop-filter), window-style panels and animations using transform and opacity only.
JavaScript ES6+	Hash routing, view rendering with template literals, form state, validation, scoring, analytics, export, sharing and storage. IntersectionObserver reveals sections on scroll.
localStorage	Forms, responses, demo profile, starred and trashed forms, template usage counts and sidebar state.
sessionStorage	Duplicate-submission prevention per session and the remember-me choice.
Clipboard and Web Share APIs	Copy-link button with a fallback, and the native share sheet where supported.
GitHub and GitHub Pages	Version control and free static hosting of the live project.

RT FORMS uses **no external JavaScript or CSS libraries** and no web fonts; the system font stack (-apple-system, SF Pro Display, Inter, system-ui) keeps it fast and easy to audit. QR codes were left for a later version because they need a library. Node.js is used only to syntax-check script.js, and browser developer tools for debugging.

CHAPTER 3 Project Design and Modules

3.1 System Architecture

Four layers, each talking only to the one below, allow storage to be replaced without rewriting the interface.

Layer	Responsibility	Examples
Presentation	Renders views, handles clicks and typing	Landing, dashboard, gallery, editor, public form
Application logic	Routing, validation, scoring, analytics, export	route(), rq(), chk(), score(), expCSV()
Data access (API)	Only code that reads or writes data	createForm, getForm, updateForm, publishForm, submitResponse, getResponses, deleteForm
Storage	Persists one JSON document	localStorage key rtforms1 (replaceable by a backend)

3.2 Pages and Navigation

Navigation uses the URL hash, so the site works on any static host. Every view has its own address.

View	Route	View	Route
Landing page	#home	Public form	#fill-ID
Login / Register	#login, #register	Form preview	#prev-ID
Dashboard / Trash	#dash	Responses	#resp-ID

View	Route	View	Route
Template gallery	#tpls	Settings / Profile	#prefs
Template preview	#tprev-N	404 page	any unknown route
Form editor	#edit-ID		

3.3 Design System

All colours are CSS variables defined once. Mint marks the brand and active navigation, lime marks primary buttons and badges, and dark green gives strong text and accents.

Primary mint	Secondary mint	Lime green	Light mint	Dark green	Dark text
#98E2C6	#72DDB5	#B7F34A	#F1FBF6	#176B50	#20352D

3.4 Modules

Module	Features
Landing page	Sticky glass navigation, animated hero with editor mockup, features, popular templates, how it works, builder preview, quiz and analytics sections, call to action and the silk footer with GitHub and Project buttons and the credit “Design & Developed by RT1504”.
Authentication (demo)	Name, e-mail, password, confirmation and terms; remember me, show password, forgot-password placeholder and a demo account. Passwords are never stored.
Dashboard	Collapsible sidebar (Dashboard, My Forms, Templates, Starred, Trash, Settings), search toolbar, animated statistics from real data, form cards with a full action menu, grid and list view, sort, filter, trash with restore, JSON import.
Template gallery and preview	15 templates in 8 categories with unique HTML/CSS thumbnails, search, category chips, sort by name or personal usage, clear filters; preview page with sample questions, estimated time, live Desktop and Mobile form; Use this template creates an independent copy.
Form builder	Editable title, description and header colour; active-question highlight; add, move, duplicate and delete; required, description, placeholder, options with Other, grid rows and columns, scale range and labels; autosave indicator; JSON export.
Form settings	Open or closed, collect e-mail, progress indicator, quiz mode, shuffle, show score and answers, passing score, confirmation and closing messages, response limit, start and end dates (browser-enforced, not secure).
Quiz	Correct answers, points and explanations; multiple choice and dropdown match one answer, checkboxes need the exact set; other types go to manual review; score stored with each response.
Respondent page	No editing controls; validation of required, e-mail, phone, number and grid rows; progress bar; open, date, limit and duplicate checks; confirmation and quiz result.
Sharing	Random form ID, link ending #fill-ID, Copy link, native Share, open and preview links, and an honest notice that links work only in the browser that holds the data.
Responses and analytics	Totals, average score, option distributions, rating averages, daily trend, score distribution, search, details, delete with confirmation, CSV (quoted cells), JSON and print report.

3.5 Question Types

Type	Settings and validation
Short answer, Paragraph	Placeholder, required

Type	Settings and validation
Multiple choice, Checkboxes, Dropdown	Options, optional Other, required, correct answer in quiz mode
Linear scale / Rating	Minimum 0 or 1, maximum 2–10, labels / stars 1–5
Date, Time, Number, Email, Phone	Native pickers; numeric, e-mail and phone (7–15 characters) validation
File upload	Placeholder explaining that storage needs a backend
Multiple choice grid / Checkbox grid	Rows and columns; one choice per row / several per row
Section heading, Info block	Display-only title and text

3.6 Template Library

No.	Template	Category	No.	Template	Category
1	Student Feedback Form	Education	9	Course Evaluation Form	Feedback
2	Teacher Evaluation Form	Education	10	Website Feedback Form	Feedback
3	Event Registration Form	Registration	11	Product Survey	Business
4	Customer Feedback Form	Customer Service	12	Bug Report Form	Customer Service
5	Online Quiz	Quizzes	13	Workshop Registration	Events
6	Student Attendance Survey	Education	14	Employee Satisfaction Survey	Feedback
7	Contact Form	Business	15	General Questionnaire	Personal
8	Job Application Form	Business			

3.7 Data Model

Object	Main fields
Form	id, title, desc, color, status, quiz, pass, showScore, showAns, start, end, star, trash, created, mod, set (open, email, confirm, closed, limit, prog, shuf), qs
Question	id, t (type), q, d, req, ph, opts, other, rows, cols, min, max, lo, hi, pts, ans, why
Response	id, at (timestamp), email, a (answers by question ID), sc (earned, max, manual)

CHAPTER 4 System Implementation

4.1 Project Structure

```
rt-forms/
├── index.html        app root, dialog and toast containers
├── style.css         design system, components, animations, responsive rules
├── script.js         router, views, builder, renderer, quiz, analytics, storage
├── README.md  ·  .gitignore
└── assets/logo/      rt-forms-logo.png, rt-forms-icon.svg, favicon.svg
```

4.2 Routing

On load the script reads saved data, restores the session choice, registers a hashchange listener and renders the current route. Unknown names show the 404 page.

```
function route() {
  const h = location.hash.slice(1) || 'home';
  const i = h.indexOf('-');
  const page = i < 0 ? h : h.slice(0, i), id = i < 0 ? '' : h.slice(i + 1);
  const views = { home, dash, tpls, edit: () => editor(id), fill: () => fill(id, 0) };
  (views[page] || nf)();
}
```

4.3 Quiz Scoring and Safe Saving

```
function score(form, answers) {
  let earned = 0, max = 0, manual = 0;
  form.qs.forEach(q => {
    if (['section', 'info'].includes(q.t)) return;
    if (!['mc', 'cb', 'dd'].includes(q.t) || !q.ans.length) { manual++; return; }
    max += q.pts; const v = answers[q.id];
    const ok = q.t === 'cb' ? Array.isArray(v) && v.length === q.ans.length && q.ans.every(x =>
v.includes(x)) : v === q.ans[0];
    if (ok) earned += q.pts;
  });
  return { s: earned, m: max, man: manual };
}

save() { try { localStorage.setItem(KEY, JSON.stringify(DB)); } catch (e) { toast('Storage full or
blocked'); } }
```

4.4 Key Functions

Function	Purpose
route(), home(), auth(), login()	Navigation, landing page and demo authentication
shell(), dash(), list(), stats()	Application shell, dashboard cards and live statistics
tpls(), tgrid(), tprev(), mkT(), useT()	Gallery, filtering, preview and creating an independent copy
ed(), qedit(), rq(), get(), chk()	Editor, question settings, rendering, reading and validating answers
score(), fill(), sub()	Quiz scoring, respondent page and submission
resp(), rl(), expCSV(), share()	Analytics, response list, CSV export and sharing
API.* and S.save()	Data-access layer and safe saving with quota handling

4.5 Deployment and Security Notes

- **Run locally:** open index.html, or serve the folder with `python3 -m http.server 8000`.
- **GitHub Pages:** push to the repository, open Settings → Pages, choose the main branch and root folder. Netlify and Vercel need no build command.
- All user text is escaped before insertion, preventing HTML injection. Imported JSON is checked and every imported form receives new IDs.
- Authentication is a demo; browser-based limits can be bypassed. Production needs server-side rules, spam protection, rate limiting and row-level security. No secret keys exist in the code.

CHAPTER 5 Software Testing

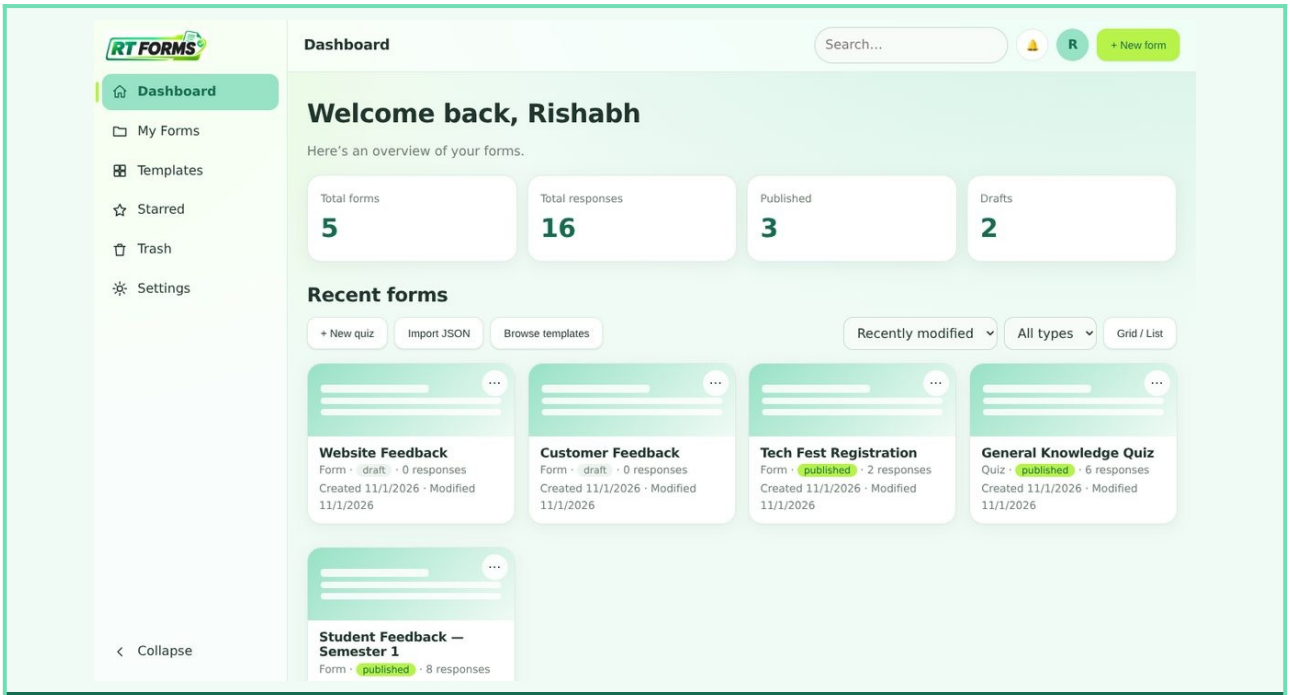
RT FORMS was checked by a syntax check of script.js (Node.js), manual functional testing in a desktop browser, and responsive testing at phone, tablet and desktop widths. The **Result** column is left for the tester to tick after running each case on the live project.

ID	Test	Expected result	Result
T01	Open the live link	Landing page loads, no console errors	☐
T02	Register with invalid e-mail or short password	Error shown, no account created	☐
T03	Press Use demo account	Dashboard opens with the user name	☐
T04	Open Templates; search and filter	15 thumbnail cards; filters work and clear	☐
T05	Preview a template, switch Desktop / Mobile	Live form shown; nothing saved	☐
T06	Use this template	New editable copy; original unchanged	☐
T07	Star, rename, duplicate, delete, restore	Each action works; Trash restores	☐
T08	Add all 17 question types	Every type renders and saves	☐
T09	Edit options, grid, scale; move and duplicate	Preview reflects every change	☐
T10	Type in a field and watch the indicator	Cursor stays; “Saved ✓” appears	☐
T11	Submit with an empty required question	Error below question; focus moves to it	☐
T12	Enter invalid e-mail, phone or number	Matching validation message	☐
T13	Submit a valid response, then again	Confirmation, then duplicate message	☐
T14	Take the quiz with right and wrong answers	Score equals points of correct answers	☐
T15	Open the response page	Charts and averages match submissions	☐
T16	Export CSV with commas, quotes, line breaks	Structure stays correct in a spreadsheet	☐
T17	Export JSON, import on dashboard	New draft with the same questions	☐
T18	View every screen at about 375 px	No sideways scroll; sidebar becomes drawer	☐
T19	Use only the keyboard	All controls reachable with visible focus	☐
T20	Enable reduced motion	Animations and counters switch off	☐

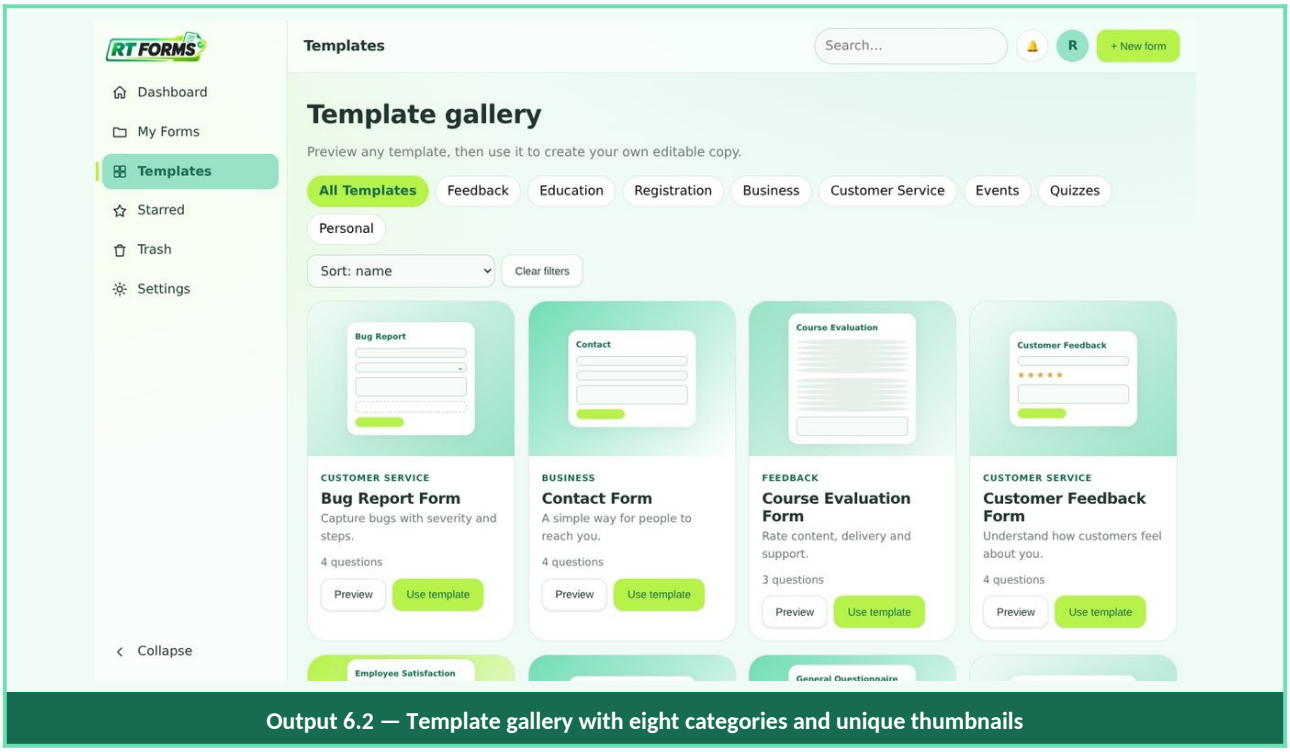
The tests cover the main journeys from sign-in to export on one browser dataset. Sharing one form between several devices is not tested because the local version does not support it.

CHAPTER 6 Output Screens

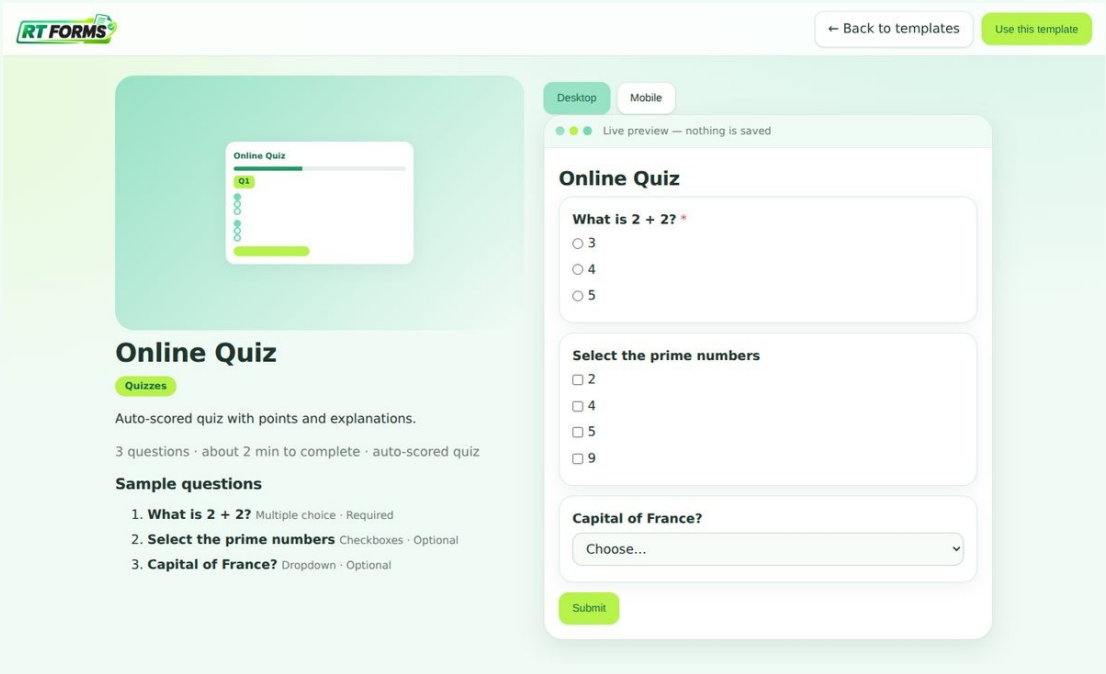
The screens below were captured from the running application with sample test data (a student feedback form with eight responses and a quiz with six responses).



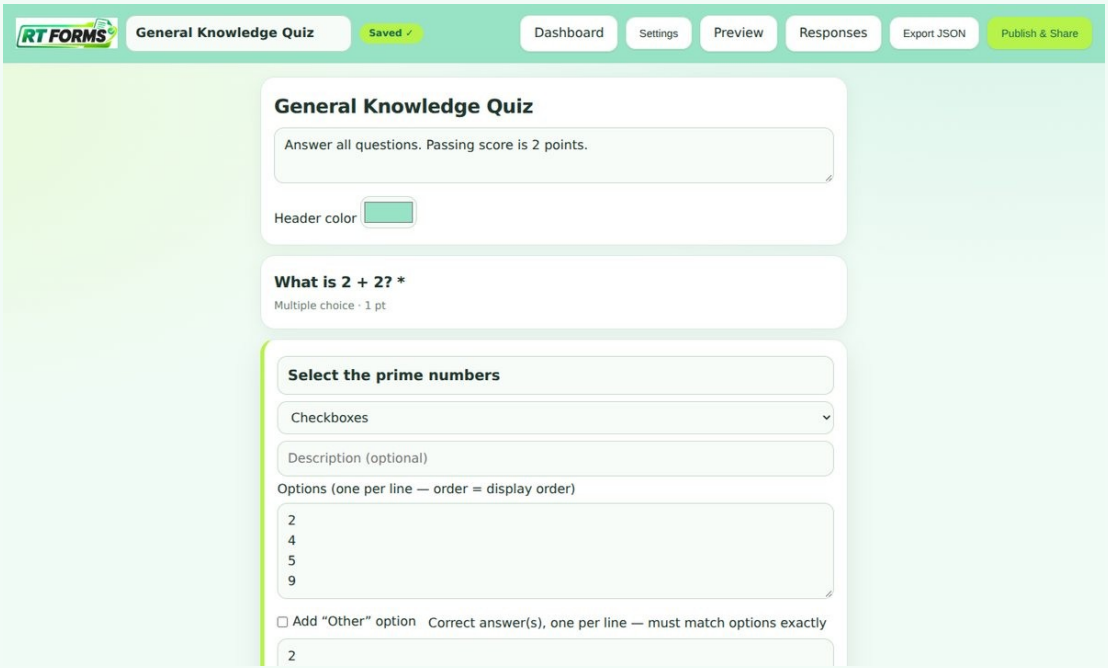
Output 6.1 — Dashboard with live statistics, form cards and sidebar



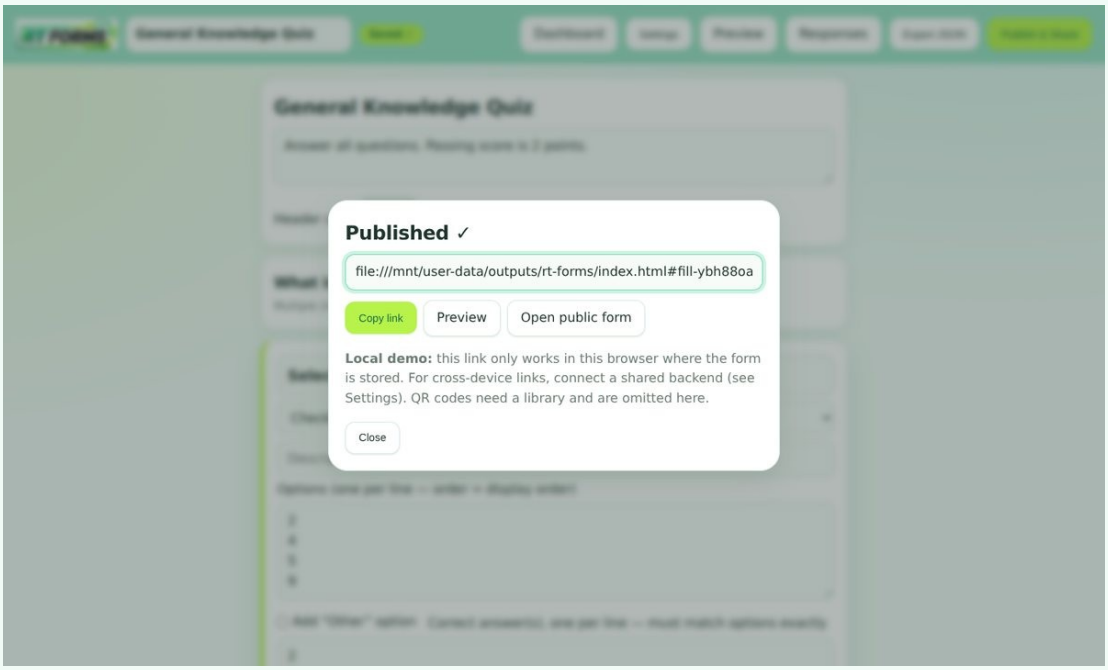
Output 6.2 — Template gallery with eight categories and unique thumbnails



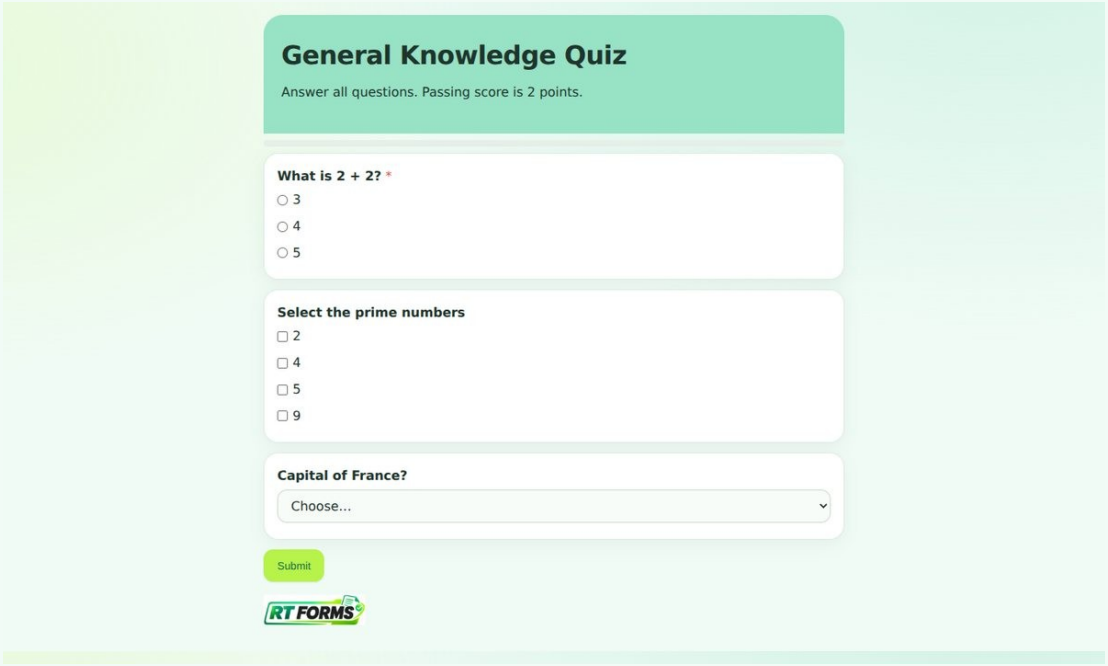
Output 6.3 — Template preview with sample questions and a live respondent form



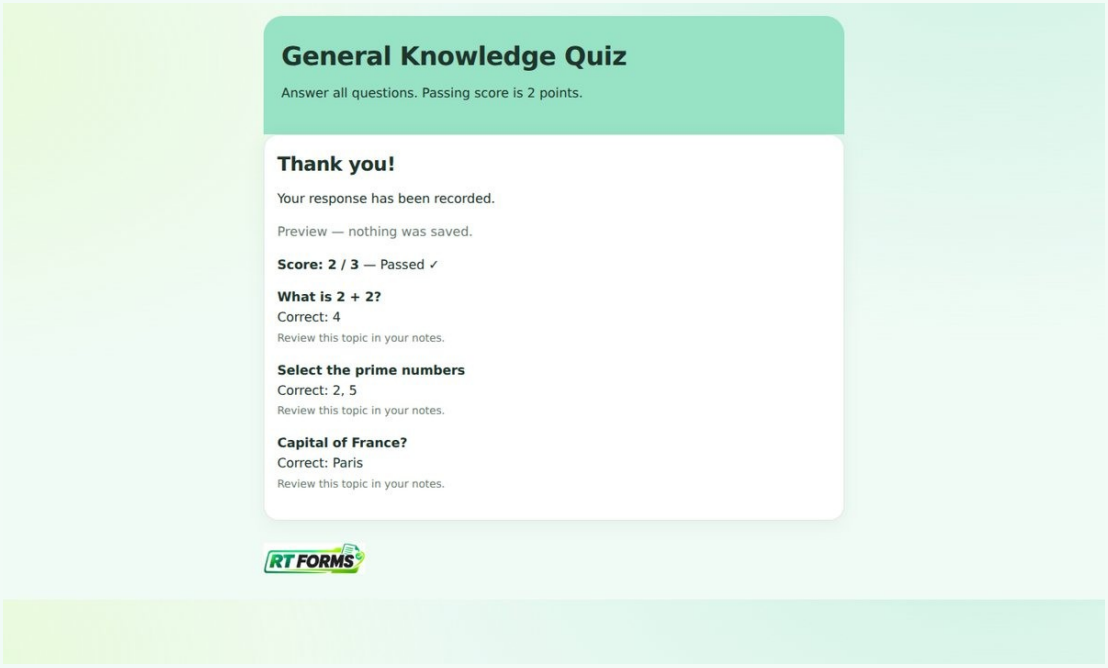
Output 6.4 — Form editor with quiz settings on the active question



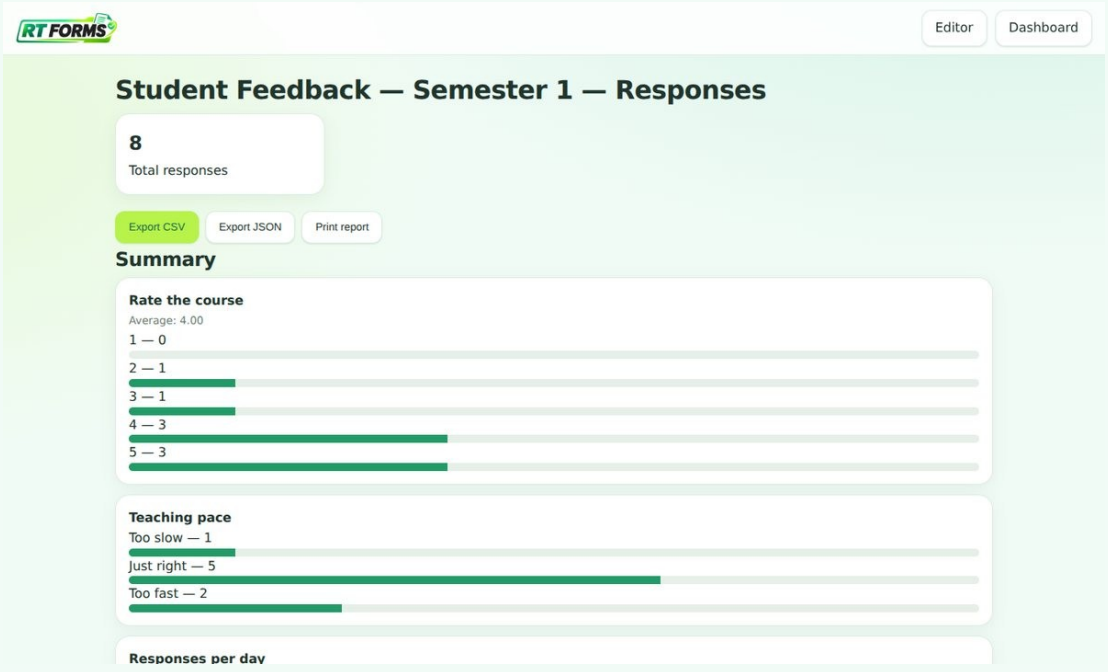
Output 6.5 — Share dialog: copy link, native share and local-storage notice



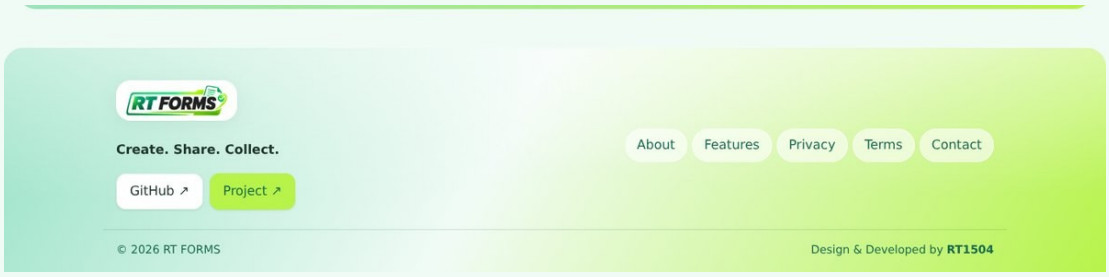
Output 6.6 — Public form page seen by a respondent



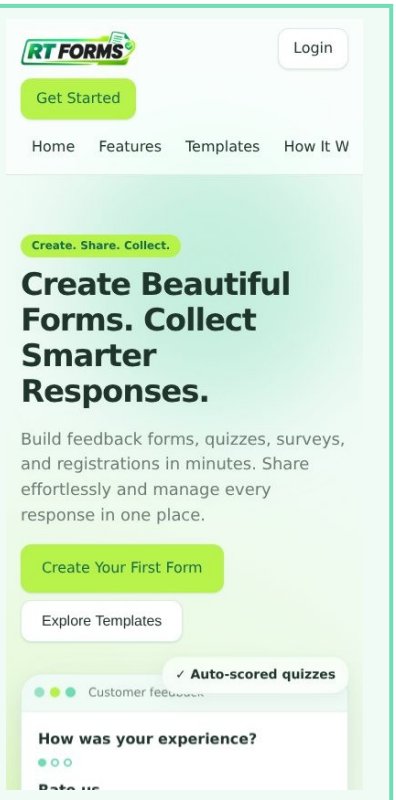
Output 6.7 — Quiz result with score, pass status and explanations



Output 6.8 — Response dashboard with rating and choice charts



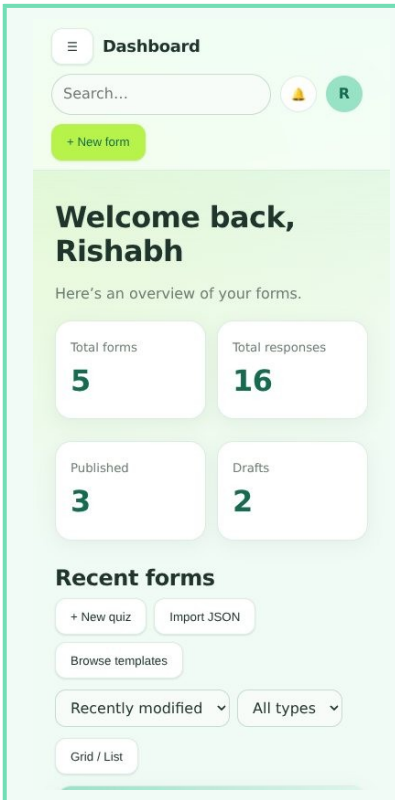
Output 6.9 — Silk footer with GitHub and Project buttons and developer credit



Output 6.10 — Mobile landing



Output 6.11 — Mobile preview



Output 6.12 — Mobile dashboard

CHAPTER 7 Learning Outcomes

7.1 Skills and Knowledge Gained

- **Architecture:** separating presentation, logic and data access so storage can change without rewriting views.
- **Routing and state:** a hash router for static hosts, one data object, safe saving and targeted re-rendering.
- **Form engineering:** rendering and validating many input types, grids and rules; quiz scoring; option distributions, averages and correct CSV generation.
- **Design and interaction:** a variable-based design system and animations that respect reduced motion.
- **Accessibility, security and deployment:** labelled controls, focus management, text escaping, honest demo authentication and publishing with GitHub Pages.

7.2 Challenges and Solutions

Challenge	Solution
Sharing a form without a backend	Unique ID in the link, a clear local-storage notice, JSON export and import, and an API layer ready for a real database
Typing in the editor lost the cursor on re-render	Inputs update data directly; the editor re-renders only after structural changes
Alert, confirm and prompt boxes can be blocked	Reusable dialogs on the native dialog element
Downloads can be blocked in some environments	Export box with Copy and Download so data is never lost
Correct CSV output	Every cell quoted, embedded quotes doubled, line breaks kept
Fifteen distinct thumbnails without image files	A small set of HTML/CSS components combined differently per template

7.3 Limitations

- Data is stored per browser, so share links work only where the form was created.
- Demo authentication is not secure; limits, dates and duplicate prevention can be bypassed in the browser.
- File upload is a placeholder; drag-and-drop, undo and redo, QR codes and a three-panel editor are not yet built.

CHAPTER 8 Conclusion

RT FORMS shows that a complete, polished form platform can be built with only HTML, CSS and JavaScript. It covers the full life cycle of a form — template, build, preview, publish, share, collect, score, analyse and export — inside a consistent macOS-inspired mint, white and lime interface. The analytics use only real stored responses, the limits of local storage are stated openly, and the API layer shows a clear path from this prototype to a real multi-user product.

Future Enhancements

Priority	Enhancement	Benefit
High	Connect Supabase or Firebase through the API layer	Real accounts and forms that open on any device
High	Secure authentication, row-level security, server validation, spam protection	Privacy, ownership, trustworthy limits

Priority	Enhancement	Benefit
Medium	QR code, drag-and-drop, undo/redo, three-panel editor, richer themes	Faster sharing and editing
Medium	Split script.js into ES modules	Easier maintenance
Low	File upload storage, e-mail notifications, collaboration, more languages	Wider use cases

CHAPTER 9 References and Glossary

9.1 References

1. MDN Web Docs — HTML, CSS, JavaScript, Web Storage, Clipboard, Web Share and dialog. https://developer.mozilla.org
2. W3C — Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/
3. GitHub Docs — GitHub Pages. https://docs.github.com/pages
4. Supabase Documentation — Authentication and Row Level Security. https://supabase.com/docs
5. Firebase Documentation. https://firebase.google.com/docs
6. OWASP — Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org

9.2 Glossary

Term	Meaning
API layer	The only code allowed to read or write stored data
Autosave	Saving every change automatically
CSV	Comma-separated values file for spreadsheets
Hash routing	Choosing the screen from the part of the address after #
JSON	Text format for structured data, used for storage and export
localStorage	Browser storage that survives closing the page
Manual review	A quiz question that cannot be scored automatically
Respondent	A person who fills in a published form
Row-level security	Database rules deciding who may read or change each row
Static hosting	Serving fixed files without server code, as GitHub Pages does

DESIGNED AND DEVELOPED BY
SHAH RISHABH ALPESHBHAI · RT1504

Live Project | Project Repository | GitHub | LinkedIn